

Automated IAM Auditor Blueprint

Author: Garrett Schwindt

Date: June 24, 2026

Project Scope: Automated Identity and Access Management (IAM) Security Compliance

1. Project Architecture Summary

The Automated IAM Auditor is designed as a cloud-native, serverless security utility to enforce the principle of least privilege. The system architecture operates on a fully automated, event-driven cycle to identify security gaps without operational overhead.

- **Compute Engine:** Built on **AWS Lambda** using Python 3.11, providing a stateless execution environment that scales effortlessly and incurs zero cost when idle.
- **Orchestration:** Managed via **Amazon EventBridge**, configured to trigger the core scanning Lambda function on a recurring cron schedule (e.g., daily or weekly).
- **Data Ingestion:** Utilizes the **AWS SDK for Python (Boto3)** to query metadata from the AWS IAM global endpoint across the organization.
- **Storage & Compliance:** Auditing results are exported as structured JSON objects and compiled into a secure, versioned **Amazon S3 Bucket** for historical analysis and regulatory compliance tracking.
- **Alerting Matrix:** Critical high-severity findings (e.g., active root access keys or long-unrotated credentials) are dispatched immediately via **Amazon Simple Notification Service (SNS)** to administrative security endpoints (Email, SMS, or Slack Webhooks).

2. Complete Executable Python Script (`lambda\_function.py`)

Below is the complete, production-ready implementation of the scanning logic designed to run natively inside an AWS Lambda environment. It programmatically scans for inactive users, old console login credentials, and unrotated access keys older than 90 days.

```
import json
import boto3
import os
from datetime import datetime, timezone

# Initialize AWS clients
iam_client = boto3.client('iam')
sns_client = boto3.client('sns')
s3_client = boto3.client('s3')
```

```

# Environment variables configured via IaC
SNS_TOPIC_ARN = os.environ.get('SNS_TOPIC_ARN')
OUTPUT_BUCKET_NAME = os.environ.get('OUTPUT_BUCKET_NAME')
MAX_KEY_AGE_DAYS = 90

def calculate_age_days(date_obj):
    if not date_obj:
        return None
    now = datetime.now(timezone.utc)
    return (now - date_obj).days

def lambda_handler(event, context):
    audit_results = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "summary": {"total_users_scanned": 0, "flagged_users_count": 0},
        "flagged_findings": []
    }

    try:
        # Paginate through all IAM users in the AWS account
        paginator = iam_client.get_paginator('list_users')
        for page in paginator.paginate():
            for user in page['Users']:
                username = user['UserName']
                audit_results["summary"]["total_users_scanned"] += 1
                user_findings = []

                # 1. Audit Password / Console Access Age
                if 'PasswordLastUsed' in user:
                    password_age = calculate_age_days(user['PasswordLastUsed'])
                    if password_age and password_age > MAX_KEY_AGE_DAYS:
                        user_findings.append({
                            "type": "STALE_CONSOLE_PASSWORD",
                            "severity": "MEDIUM",
                            "message": f"User console password was last used {password_age} days ago."
                        })
    except Exception as e:
        # Handle exceptions
        pass

```

```

# 2. Audit API Access Keys
key_paginator = iam_client.get_paginator('list_access_keys')
for key_page in key_paginator.paginate(UserName=username):
    for key in key_page['AccessKeyMetadata']:
        if key['Status'] == 'Active':
            key_age = calculate_age_days(key['CreateDate'])
            if key_age and key_age > MAX_KEY_AGE_DAYS:
                user_findings.append({
                    "type": "UNROTATED_ACCESS_KEY",
                    "severity": "HIGH",
                    "id": key['AccessKeyId'],
                    "message": f"Active access key {key['AccessKeyId']} is {key_age}
days old."
                })

```

```

# Append findings if violations were uncovered
if user_findings:
    audit_results["summary"]["flagged_users_count"] += 1
    audit_results["flagged_findings"].append({
        "username": username,
        "user_arn": user['Arn'],
        "violations": user_findings
    })

```

```

# Save audit artifact to Amazon S3
file_key =
f"audit-logs/{datetime.now(timezone.utc).strftime('%Y-%m-%d')}-iam-report.json"
s3_client.put_object(
    Bucket=OUTPUT_BUCKET_NAME,
    Key=file_key,
    Body=json.dumps(audit_results, indent=2, default=str),
    ContentType='application/json'
)

```

```

# Fire alerts via Amazon SNS if critical high-severity items exist
high_severity_count = sum(
    1 for item in audit_results["flagged_findings"]
    for v in item["violations"] if v["severity"] == "HIGH"
)

```

```

    if high_severity_count > 0 and SNS_TOPIC_ARN:
        alert_message = (
            f"ALERT: Automated IAM Auditor discovered {high_severity_count} high-severity
violations.\n"
            f"Report generated at: {audit_results['timestamp']}\n"
            f"Artifact Destination: s3://{OUTPUT_BUCKET_NAME}/{file_key}\n"
            f"Please review immediately to restrict over-privileged access patterns."
        )
        sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject="CRITICAL: IAM Compliance Auditor Violations Found",
            Message=alert_message
        )

    return {
        "statusCode": 200,
        "body": json.dumps({"message": "Audit completed successfully",
"findings_uncovered": len(audit_results["flagged_findings"])}))
    }

except Exception as e:
    print(f"Error executing security audit: {str(e)}")
    raise e

```